

Compilateur du Langage NilNovi Procédural (NNP)

Rapport de projet de Théorie des Langages et Compilation

**Julien Bourdet
Malo Joannon
Axel Pavageau**

INFO2
ENSSAT Lannion

Mai 2026

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Architecture Globale du Compilateur	3
2	Analyses Lexicale et Syntaxique	4
2.1	Analyse Lexicale (<code>analex.py</code>)	4
2.2	Analyse Syntaxique (<code>anasyn.py</code>)	4
3	Table des Identificateurs	5
3.1	Structure Interne de la Table	5
4	Analyse Sémantique	6
4.1	Vérification des Déclarations et de la Portée	6
4.2	Gestion des Types	6
4.3	Initialisation avant Utilisation	6
4.4	Validation des Modes de Paramètres	6
5	Génération de Code	7
5.1	Rôle du Générateur	7
5.2	Traduction des Structures de Contrôle	7
5.2.1	Génération d'une alternative (<code>if-then-else</code>)	7
6	Machine Virtuelle	8
6.1	Architecture de la Machine Virtuelle	8
6.2	Jeu d'Instructions	8
6.3	Protocole d'Appel de Procédure / Fonction	9
6.4	Retour de fonction et restauration	9
7	Difficultés, Perspectives et Conclusion	10
7.1	Difficultés rencontrées	10
7.2	Perspectives d'Évolutions	10
7.3	Conclusion	10

1 Introduction

1.1 Contexte

Dans le cadre de l'unité d'enseignement de Théorie des Langages et Compilation (TLC), ce projet consiste à réaliser un compilateur complet pour le langage algorithmique **NILNOVI PROCÉDURAL (NNP)** ainsi qu'une **Machine Virtuelle à Pile (MV)** pour en exécuter le code objet.

Le langage NNP est un langage impératif et fortement typé inspiré de la syntaxe du langage Ada. Il dispose d'instructions classiques d'affectation, de branchements conditionnels (**if-then-else**), de boucles (**while-loop**), d'entrées-sorties de base (**get** et **put**), ainsi que du support de la déclaration et de l'appel de procédures et de fonctions avec passage de paramètres par valeur (**in**) et par référence (**in out**).

1.2 Architecture Globale du Compilateur

La structure du compilateur réalisée est illustrée par la figure 1. Elle s'articule autour de cinq modules distincts et complémentaires écrits en Python :

- **L'analyseur lexical** (`analex.py`) qui convertit le flux de caractères source en unités lexicales (tokens).
- **L'analyseur syntaxique récursif descendant** (`anasyn.py`) qui pilote l'ensemble des étapes et assure l'application de la grammaire LL(1).
- **La table des identificateurs** (`IdentifierTable` imbriquée dans `anasyn.py`) qui gère la portée des variables et leur adressage.
- **L'analyseur sémantique** (`anasem.py`) qui réalise les vérifications de types, d'initialisation et de signature des appels.
- **Le générateur de code** (`code_generator.py`) qui émet les instructions en langage intermédiaire.
- **La machine virtuelle à pile** (`vm.py`) qui exécute le code objet généré.

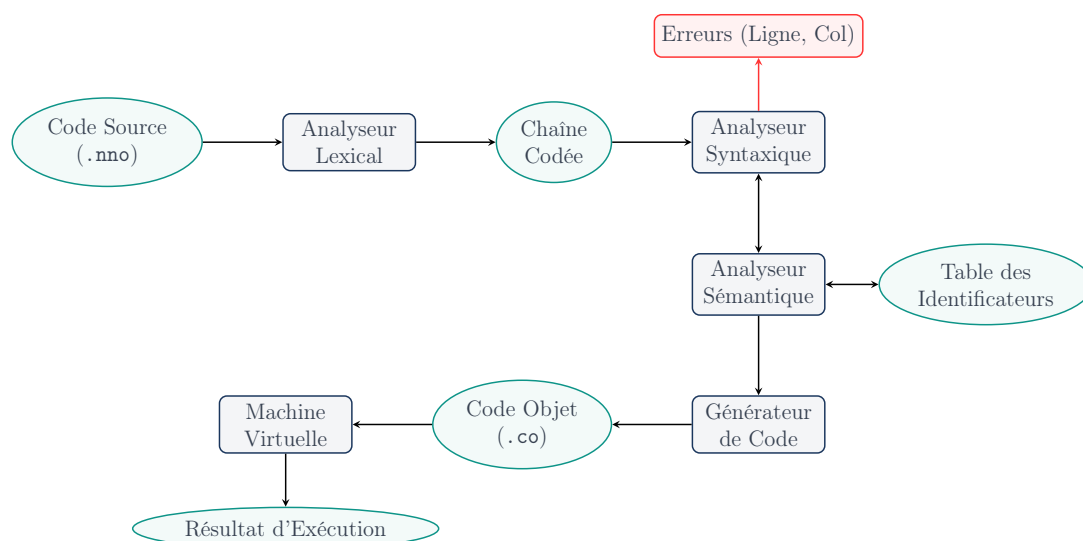


FIGURE 1 – Architecture modulaire du compilateur NNP et flux de traitement

2 Analyses Lexicale et Syntaxique

2.1 Analyse Lexicale (`analex.py`)

L'analyseur lexical convertit les lignes de texte source en une suite d'instances de la classe `LexicalUnit`. Il ignore les commentaires (débutant par `//`) ainsi que les espaces et tabulations. Le langage NNP comprend 6 unités lexicales principales modélisées par héritage :

- **Keyword** (mot-clé) : représente les 24 mots-clés du langage tels que `procedure`, `if`, `while`, `integer`.
- **Identifiant** (identificateur) : chaînes alphanumériques représentant les variables ou opérations.
- **Integer** : stocke la valeur numérique entière lue.
- **Character** : les caractères délimiteurs de ponctuation simples (`;`, `:`, `,`, `(`, `)`).
- **Symbol** : opérateurs composés ou de comparaison (`:=`, `<=`, `>=`, `/=`, `=`).
- **Fel** : l'unité fictive représentant le point final `.` achevant le programme principal.

2.2 Analyse Syntaxique (`anasyn.py`)

L'analyse syntaxique valide la conformité grammaticale à l'aide d'un analyseur prédictif récursif descendant.

Chaque fonction de grammaire (telle que `program`, `corpsProgPrinc`, `partieDecla`, `instr`, `expression`, etc.) consomme les tokens fournis par `LexicalAnalyser` en utilisant des assertions comme `acceptKeyword(k)` ou `acceptCharacter(c)`. En cas de divergence par rapport aux schémas autorisés par les diagrammes, une exception `AnaSynException` est levée.

3 Table des Identificateurs

3.1 Structure Interne de la Table

La table des identificateurs est implémentée à l'aide d'une classe `IdentifierTable`. La portée des variables (globales, locales) est gérée par un dictionnaire imbriqué, où chaque clé représente une portée et la valeur associée est un autre dictionnaire contenant les variables déclarées dans cette portée.

```
1 class IdentifierTable:
2     def __init__(self):
3         self.table = {"global": {}}
4         self.address_counters = {"global": 0}
```

Listing 1 – Structure de base de la Table des Identificateurs

Chaque variable déclarée y est enregistrée avec des caractéristiques : son type (`integer` ou `boolean`), son rôle (`variable`, `parameter`, ou `local variable`), son adresse d'accès en mémoire ainsi que le mode du paramètre (`in` ou `in out`).

Un compteur d'adresse indépendant est incrémenté pour chaque portée à chaque ajout de variable, garantissant l'absence de chevauchement d'adresses au sein d'un même espace de portée.

4 Analyse Sémantique

L'analyseur sémantique est chargé de s'assurer que le programme respecte les contraintes logiques et les règles de typage du langage NNP. Ce composant est défini dans la classe `SemanticChecker` (`anasem.py`) et effectue plusieurs contrôles cruciaux en temps réel durant le parcours syntaxique.

4.1 Vérification des Déclarations et de la Portée

À l'aide d'une pile de dictionnaires représentant les blocs (`self.scopes`), le sémantique vérifie :

- Qu'une variable n'est pas déclarée deux fois au sein de la même portée (`declare_variable`).
- Qu'une variable ou fonction accédée a bien été préalablement déclarée (`check_variable_declared`).

4.2 Gestion des Types

Le compilateur NNP supporte deux types primitifs : `integer` (entier) et `boolean` (booléen). Les vérifications suivantes sont systématiquement opérées :

- **Opérations et Expressions** : les opérateurs arithmétiques (+, -, *, /) et de comparaison exigent des opérandes de type `integer` et renvoient respectivement un type `integer` ou `boolean`. Les opérateurs logiques (`and`, `or`, `not`) imposent des opérandes de type `boolean`.
- **Conditions** : Les expressions régissant les conditions d'un `if` ou d'une boucle `while` doivent impérativement s'évaluer en un type `boolean` (`check_condition_if` et `check_condition_while`).
- **Affectations** : Le type de la valeur affectée doit concorder exactement avec le type déclaré de la variable cible.

4.3 Initialisation avant Utilisation

Pour éviter les comportements indéterminés à l'exécution, la table des symboles conserve un booléen d'état `initialized` :

- Une affectation ou une opération `get` marque la variable comme initialisée.
- Toute lecture d'une variable dans une expression provoque un appel à `check_init_variable`, levant une erreur si la variable n'a pas encore reçu de valeur.

4.4 Validation des Modes de Paramètres

Le langage NNP intègre deux modes d'appel pour les sous-programmes :

- **Mode in** : Passage par valeur. La variable locale est lue mais ne peut pas être réécrite par l'appelant en retour.
- **Mode in out** : Passage par référence (adresse). L'appelant transmet l'adresse du paramètre. Toute modification au sein du sous-programme affecte directement la variable d'origine. L'analyse sémantique s'assure ainsi que le paramètre transmis à un argument `in out` est bien une variable modifiable et non une expression ou une constante.

5 Génération de Code

Le module `code_generator.py` fournit la classe `CodeGenerator` chargée de construire la liste d'instructions intermédiaires, nommée *code pseudo-assembleur objet*, et de gérer la résolution d'adresses.

5.1 Rôle du Générateur

L'analyseur syntaxique pilote la génération de code en appelant directement la méthode `emit(opName, arg)` au sein de ses fonctions de traitement grammatical. Le format du code objet produit utilise une écriture standardisée de type `instruction(argument)` ou `instruction()`, ce qui facilite son chargement par la Machine Virtuelle.

5.2 Traduction des Structures de Contrôle

La traduction des structures de contrôle comme `if` et `while` nécessite l'utilisation d'instructions de branchement inconditionnel (`tra(adresse)`) et conditionnel si faux (`tze(adresse)`). Comme l'adresse de destination d'un branchement n'est pas connue au moment de sa première rencontre, le compilateur réalise un **backpatching** (remplissage d'adresses a posteriori).

5.2.1 Génération d'une alternative (if-then-else)

1. Évaluation de l'expression conditionnelle (qui empile 0 ou 1).
2. Émission d'un `tze` avec un argument temporaire `None`. On mémorise la position de ce `tze` dans le code objet.
3. Traitement du bloc `then`.
4. S'il y a un `else` :
 - Émission d'un `tra(None)` pour sauter la partie `else` à la fin du `then`.
 - Remplacement de l'argument du `tze` mémorisé par l'adresse de début du bloc `else`.
 - Traitement du bloc `else`.
 - Résolution de l'adresse de saut du `tra` par l'adresse courante à la sortie du bloc `else`.
5. S'il n'y a pas de `else` : Résolution de l'adresse du `tze` par l'adresse courante à la fin de l'alternative.

6 Machine Virtuelle

La Machine Virtuelle (MV), implémentée dans `vm.py`, est un interpréteur à pile d'exécution linéaire. Elle reproduit le comportement matériel d'une architecture bas niveau.

6.1 Architecture de la Machine Virtuelle

La structure interne de la MV est caractérisée par les registres et espaces mémoires suivants :

- **stack** : La pile d'exécution contenant les valeurs (entiers, booléens) et les cadres d'activation.
- **base** : Pointeur d'accès au cadre (bloc d'activation) courant pour la gestion des variables locales et paramètres.
- **pc** (Program Counter) : Le compteur ordinal pointant vers l'instruction courante du code objet.

6.2 Jeu d'Instructions

La machine virtuelle NNP exécute 31 instructions bas niveau classées comme suit :

- **Gestion de pile et mémoire** : `reserver(n)`, `empiler(val)`, `empilerAd(rel_addr)` (empile l'adresse absolue calculée comme $\text{base} + 2 + \text{rel_addr}$), `affectation()` (consomme adresse et valeur et stocke la valeur dans la pile à l'adresse indiquée), `valeurPile()` (remplace l'adresse au sommet de la pile par la valeur contenue à cette adresse).
- **Opérations mathématiques** : `add`, `sous`, `mult`, `div`, `moins` (négation arithmétique).
- **Opérations logiques et comparaisons** : `et`, `ou`, `non`, `egal`, `diff`, `inf`, `infeg`, `sup`, `supeg`.
- **Contrôle et Branchements** : `tra(pc_dst)`, `tze(pc_dst)`, `debutProg`, `finProg`, `erreur(msg)`.
- **Entrées-Sorties** : `get()` (lit un entier sur le terminal et le stocke à l'adresse dépilée), `put()` (affiche la valeur présente au sommet de la pile).
- **Procédures et Fonctions** : instructions d'appel et de retour (`reserverBloc`, `traStat`, `retourProc`, `retourFonct`, `empilerParam`).

6.3 Protocole d'Appel de Procédure / Fonction

La mise en place de sous-programmes exige de structurer rigoureusement les blocs d'activation de la pile. Les étapes suivantes sont coordonnées par le compilateur et la MV lors d'un appel :

1. **Préparation** : L'instruction `reserverBloc` insère deux cases réservées au sommet de la pile d'exécution : la sauvegarde de l'ancienne valeur du registre `base` (chaînage dynamique) et la case pour l'adresse de retour (sauvegarde du `pc`).
2. **Empilement des paramètres** : L'appelant calcule et empile les paramètres (`in` ou `in out`) définis.
3. **Saut et Liaison** : L'instruction `traStat(addr, nbp)` met à jour le registre `base` pour cibler le début du nouveau bloc d'activation de la fonction, sauvegarde le `pc` courant dans l'emplacement d'adresse de retour, puis effectue le saut vers la première instruction du sous-programme (`pc = addr`).
4. **Allocation locale** : Le sous-programme alloue l'espace pour ses variables locales via `reserver(n)`.

Le schéma de la pile virtuelle lors de l'exécution d'une fonction avec paramètres est représenté en figure 2.

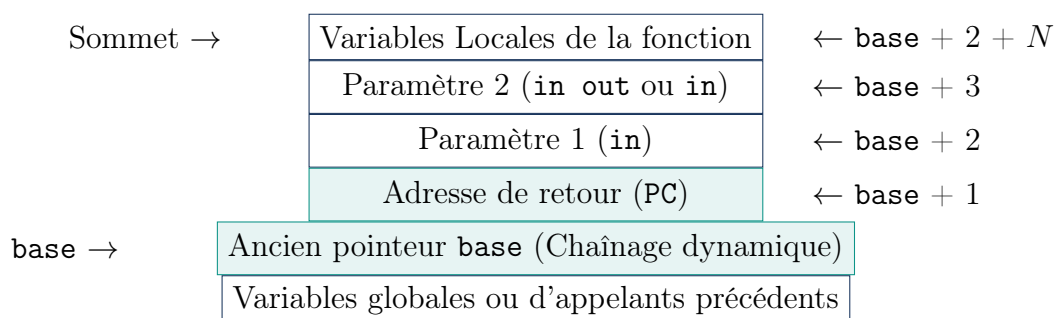


FIGURE 2 – Organisation du bloc d'activation de la pile lors d'un appel

6.4 Retour de fonction et restauration

Lorsqu'un sous-programme s'achève, la restauration de l'état précédent de la MV s'effectue comme suit :

- **Procédure (retourProc)** : Restaure la valeur de `base` lue à `stack[base]`, réassigne `pc` à la valeur de retour stockée à `stack[base + 1]`, puis nettoie l'ensemble du bloc d'activation en supprimant les éléments de la pile à partir de `base` (`del stack[base:]`).
- **Fonction (retourFonct)** : Récupère au préalable la valeur de retour au sommet de la pile, effectue la même restauration de `base` et `pc` avec nettoyage de la pile, puis ré-empile la valeur de retour finale pour l'appelant.

7 Difficultés, Perspectives et Conclusion

7.1 Difficultés rencontrées

La conception de ce compilateur a principalement été marquée par la complexité de l'adressage indirect pour le passage de paramètres par référence (`in out`) et la mise en place du *backpatching* pour le branchement des structures conditionnelles et des boucles. L'intégration avec la machine virtuelle et la détection d'anomalies ont pu être grandement facilitées grâce au développement d'un mode de débogage pas à pas affichant dynamiquement l'état de la pile et des registres.

7.2 Perspectives d'Évolutions

- **Ajout de types complexes** : Intégration de types tels que des tableaux statiques ou des nombres flottants.
- **Optimisation de code** : Détection de code mort ou réduction d'expressions constantes à la compilation pour économiser les cycles de la MV.
- **Garbage Collector** : Gestion automatique d'une zone de tas (heap) pour de l'allocation dynamique.

7.3 Conclusion

En conclusion, ce projet a permis de concrétiser les notions théoriques du cours de TLC en concevant une chaîne de compilation complète ainsi qu'une machine virtuelle à pile fonctionnelle. L'architecture modulaire adoptée a facilité le développement et offert une excellente introduction pratique à la gestion mémoire et à l'exécution bas niveau.